Outline

Part I: Introduction to machine

- **Neural Network (NN)**
- **Convolutional Neural Network (CNN)**
- **Recurrent Neural Network (RNN)**

Part II: Applications with machine learning

- **Classification - Common CNN models**
- **Object detection and localization**
- **Object segmentation**
- **Object identification**
- **Object tracking (VOT, VOS)**

Object Identification

- DeepFace
- DeepID
- FaceNet
- Tuplet loss
- CosFace
- ArcFace

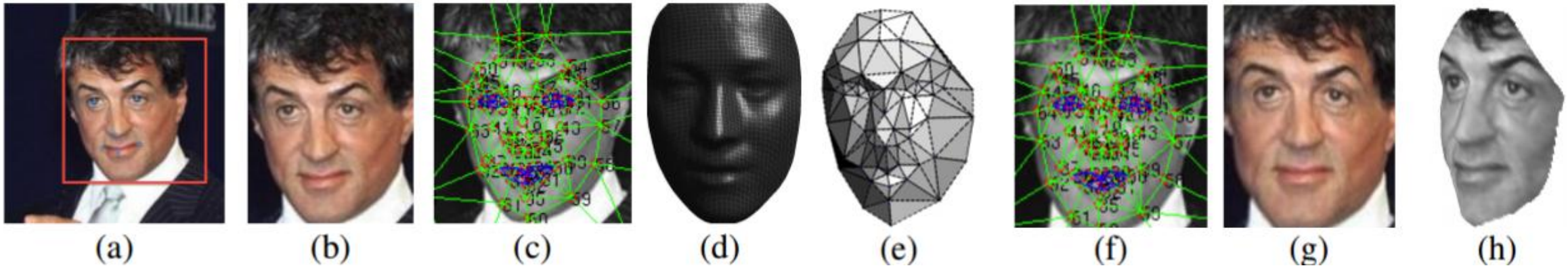
Face Recognition: DeepFace (By Meta)

- Characteristics
  - 3D face alignment
    - warp a facial crop to a 3D frontal mode by using a 3D-face modeling based on fiducial points
  - New DNN architecture for feature extraction
    - using Local Conv. layer - leverage the importance of face alignment
    - Need a large size of training dataset (due to local conv. layers)

- 3 steps: 1. face alignment
  2. face representation (feature extracted from DNN)
  3. face verification (pair matching)

Taigman Y, Yang M, Ranzato M A, et al. Deepface: Closing the gap to human-level performance in face verification

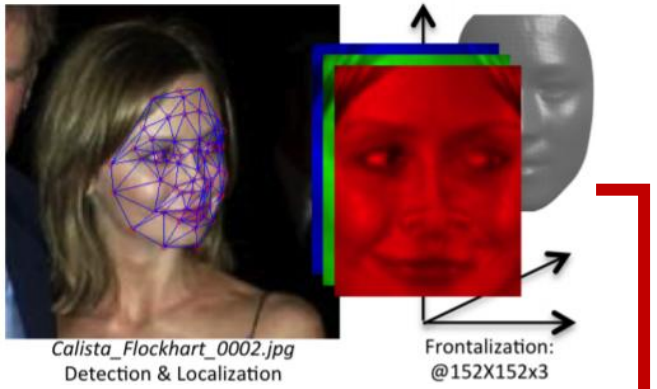
DeepFace – 3D face alignment



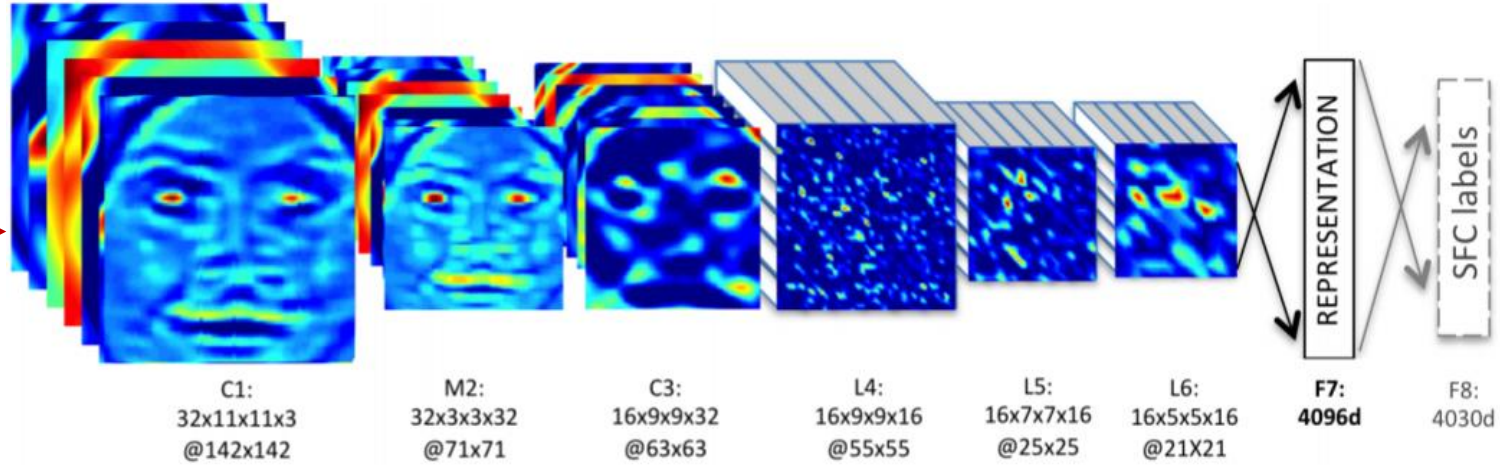
a) detect 6 fiducial points
b) transform image to fit the fiducial points into anchor points → a 2D aligned crop
c) localizing additional 67 fiducial points
d) Transform to the reference 3D shape.
e) Triangle visibility
f) Affine warping using the 67 fiducial points from the 3D model
g) The final frontalized crop
h) A new view generated by the 3D model

Taigman Y, Yang M, Ranzato M A, et al. Deepface: Closing the gap to human-level performance in face verification

DeepFace – Face representation (using DNN)



- **Only one max pooling layer**
  spatial invariant is not needed due to face alignment
- **Local-conv**
  every location in feature map learns a different set of filters (weights not shared)
- **Total params**: > 120M
  (95% from local-conv and fc)



Input: 152x152 RGB

Conv11-32

Max-pooling (3x3 s=2)

Conv9-16

Local-Conv9-16

Local-Conv7-16

Local-Conv5-16

FC-layer: 4096-d

SFC: 4030-d

Scores for 4030 classes (**face id**)
→ input to **softmax**

DeepFace – Face representation (using DNN)

- **Local Conv. layer**
  1. Weights not sharing
     - different regions of an aligned image have different local statistics
     - leverage the importance of face alignment
  2. lots # of weights
     - need much more data in training for convergence (easy to get from Facebook!)
- **FC-layer**
  - Drop-out is applied only to FC-layer (F7)
  - output of fc-layer is used as face representation feature vector
  - Feature vector normalization (to be in the range of [0, 1])
    - Each component of the vector is divided by its largest value across training set.
    - then the vector is L2-normalized ➔ x = [x1, x2, ...] ➔ $\hat{x} = \frac{x}{\|x\|_2}$.

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2}.$$

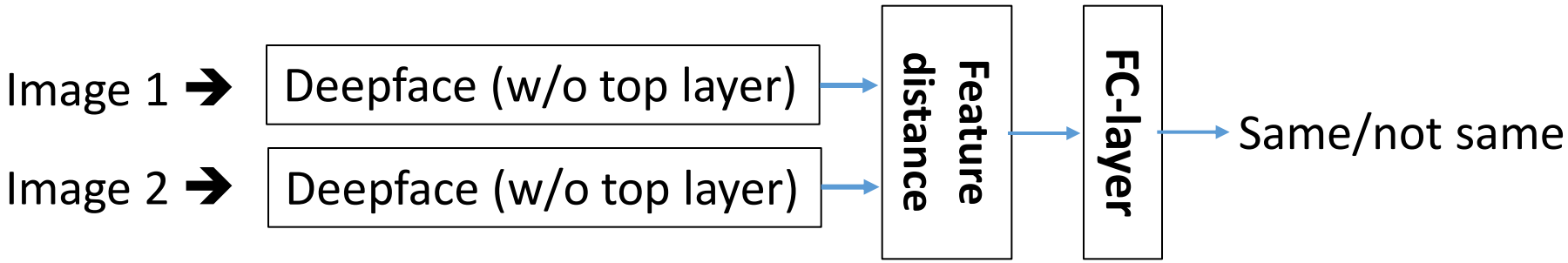
➔ new vector $\hat{x}$ has unit length:

DeepFace – Face verification

- **Face verification**: to see if two face images belonging to the same person
  - Convert face images to face representation **(**face **embeddings)**
    - Deepface without top layer: 4090d vectors
    - After training, the model can produce embeddings for any face, even ***unseen*** ones
  - Compute distance between face embeddings

$$d(f_1, f_2) = \sum_i \alpha_i |f_1[i] - f_2[i]|$$

where α_i is trainable parameter



DeepFace – Dataset

- **SFC** (Social Face Classification): photos from Facebook
  - 4.4M labeled faces from **4,030** people, each with 800 to 1200 faces,
- **LFW** (Labeled Faces in the Wild): web photos of celebrities
  - 13,323 photos from 5,749 people, divided into 6,000 face pairs
  - **Restricted**: only same/not same labels are available in training
  - **Unrestricted**: additional training pairs is accessible (By providing knowledge about the identities in training sets, hence can generate more pairs in training set
- **YTF** (YouTube Faces): video clips
  - 3,425 videos of 1,595 people, divided into 5,000 video pairs

SFC photos have larger variations in image quality, lighting, and expressions than the images of celebrities in LFW and YTF which were taken by professional photographers.

DeepFace – Result

- Ensembles of DNNs

  Combine multiple networks trained by feeding different types of inputs

  a) 3D-aligned RGB images
  b) gray-level image plus image gradient magnitude and orientation
  c) 2d-aligned RGB images

DeepFace-ensemble:

combine the results of models a, b and c, using a non-linear SVM

DeepFace-single:

the model a

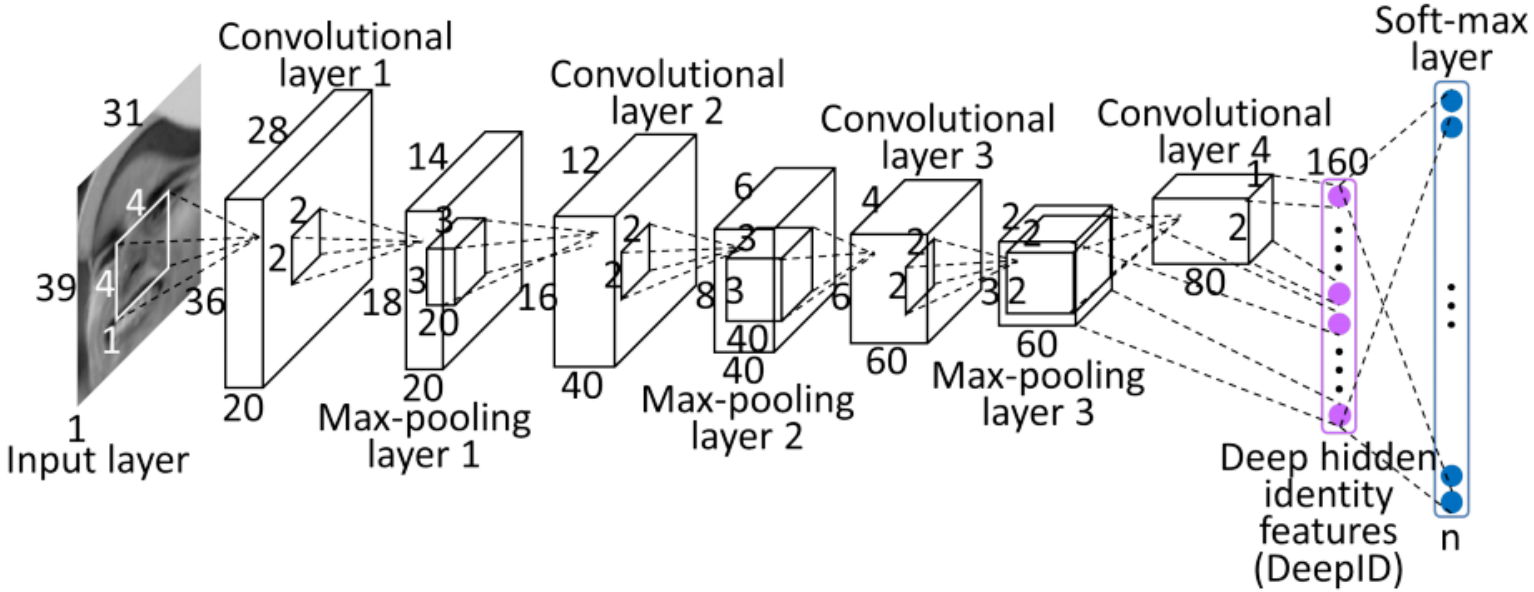
LFW (Labeled Faces in the Wild)

Method	Accuracy ± SE	Protocol
Joint Bayesian [6]	0.9242 ±0.0108	restricted
Tom-vs-Pete [4]	0.9330 ±0.0128	restricted
High-dim LBP [7]	0.9517 ±0.0113	restricted
TL Joint Bayesian [5]	0.9633 ±0.0108	restricted
DeepFace-single	**0.9592** ±0.0029	unsupervised
DeepFace-single	**0.9700** ±0.0028	restricted
DeepFace-ensemble	**0.9715** ±0.0027	restricted
DeepFace-ensemble	**0.9735** ±0.0025	unrestricted
Human, cropped	0.9753	

YTF (Youtube Faces)

Method	Accuracy (%)	AUC	EER
MBGS+SVM- [31]	78.9 ±1.9	86.9	21.2
APEM+FUSION [22]	79.1 ±1.5	86.6	21.4
STFRD+PMML [9]	79.5 ±2.5	88.6	19.9
VSOF+OSS [23]	79.7 ±1.8	89.4	20.0
DeepFace-single	**91.4** ±1.1	96.3	8.6

DeepID



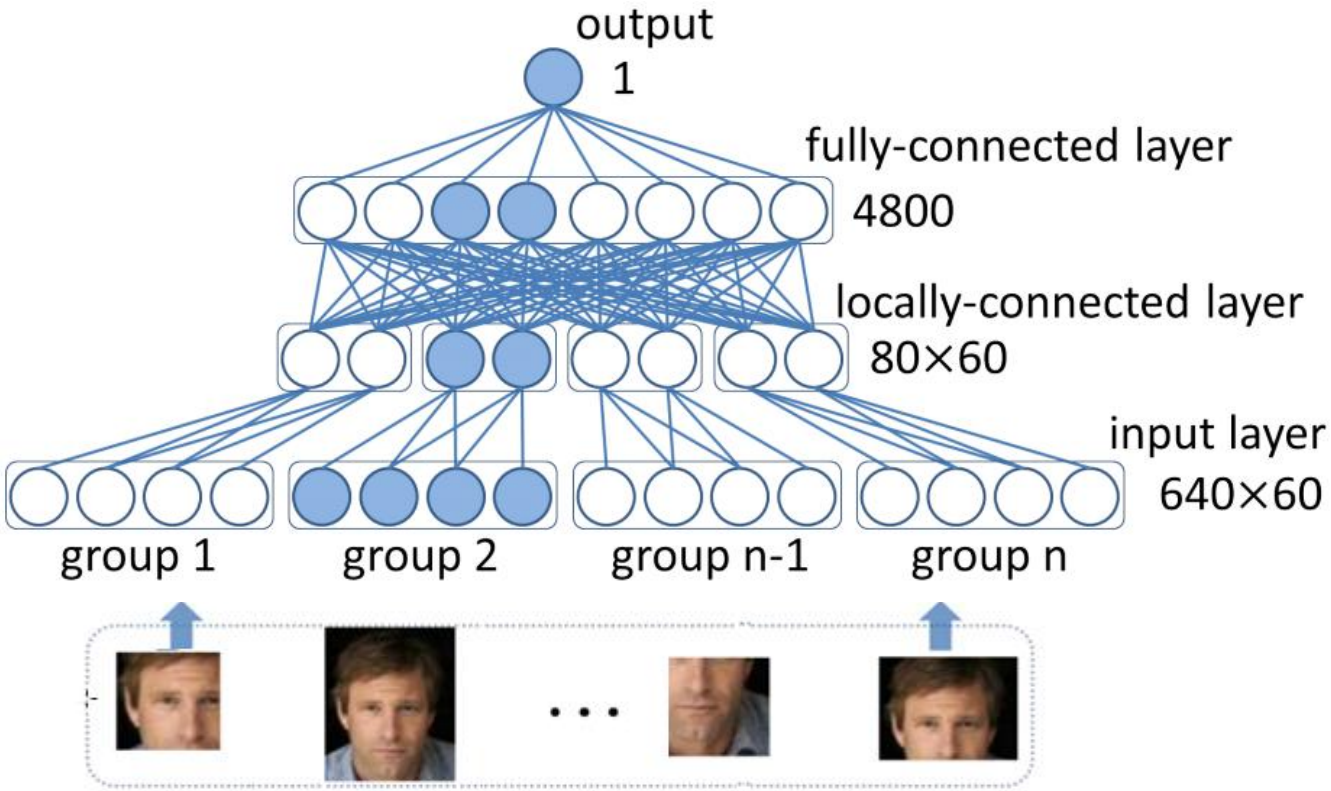
- It takes 39×31×1 input and predicts n identity classes.
- Layer 3: locally shared layer (weights are locally shared in every 2×2 regions)
- Layer 4: locally connected layer (weights are totally NOT shared).
- The dimension of the DeepID layer is fixed to 160
- DeepID layer is fully connected to both layer 3 and layer 4.
- softmax: predict the probability distribution over n different identities
  - **Identification loss** (cross-entropy loss)

DeepID

Ten regions

- **Feature extraction**
  - Faces are globally aligned
  - Features extracted from 60 patches: 10 regions, 3 scales, RGB/gray → (10x3x2)
  - trained 60 ConvNets: each extracts DeepID of 160-d x 2 (patch & flipped one)
  - Face representation: 19200-d vectors (160×2×60)

DeepID



- Face verification network
  - Input: 60 groups, each contains features from a patch pair with a particular ConvNet
  - locally-connected layer: Each neuron only connects to a single group of features
  - Fully-connected layer: to learn global relations
  - A single output: fully connected to 4800 neurons → Sigmoid
- Experimental results
  - **LFW dataset: 97.45%**

DeepID2

- **Identification loss** (cross-entropy loss)

$$\mathrm{Ident}(f, t, \theta_{id}) = -\sum_{i=1}^{n} -p_i \log \hat{p}_i = -\log \hat{p}_t$$

- **Contrastive loss** (L2 distance)
  - pulls together positive pairs and pushes apart negative pairs

$$\mathrm{Verif}(f_i, f_j, y_{ij}, \theta_{ve}) = \begin{cases} \frac{1}{2} \|f_i - f_j\|_2^2 & \text{if } y_{ij} = 1 \quad \text{(same identity)} \\ \frac{1}{2} \max\left(0, m - \|f_i - f_j\|_2\right)^2 & \text{if } y_{ij} = -1 \quad \text{(diff identity)} \end{cases}$$

m : distance margin for diff identities
Θ_v :parameters to be learned

- **DeepID loss**

Identification loss (cross-entropy) + λ Contrastive loss (L2)

DeepID2

Contrastive loss	L2	L2+	L2-	L1	cosine	none
L2 norm (%)	94.95	94.43	86.23	92.92	87.07	86.43
Joint Bayesian (%)	95.12	94.87	92.98	94.13	93.38	92.73

L2+ : only consider distances between the same identities
L2- : only consider distances between of different identities

Face verification on LFW.

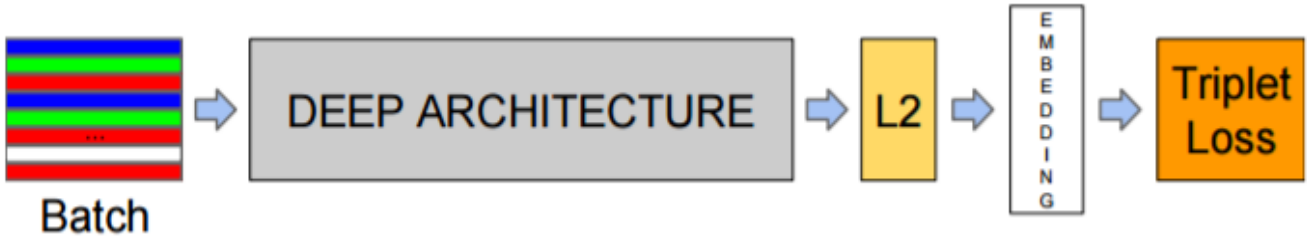
method	accuracy (%)
High-dim LBP [7]	95.17 ± 1.13
TL Joint Bayesian [5]	96.33 ± 1.08
DeepFace [31]	97.35 ± 0.25
DeepID [29]	97.45 ± 0.26
GaussianFace [23]	98.52 ± 0.66
DeepID2 [28]	99.15 ± 0.13

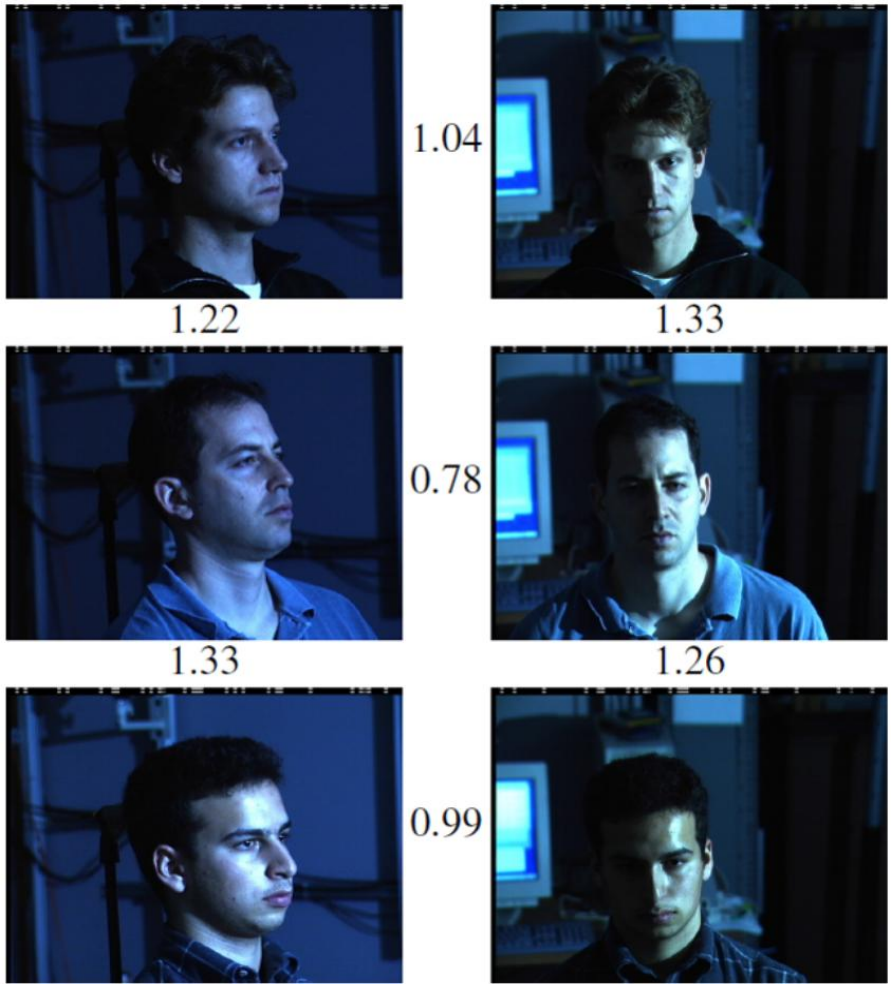
Result: → contrastive loss is mainly reducing intra-personal variations

FaceNet: *A unified embedding for face recognition and clustering*

(by Google)

- **Model structure**
  - consists of a batch input layer, a deep CNN, followed by L2 normalization, which results in the face embedding. This is followed by the triplet loss during training.
  - No softmax in CNN
  - each face is compactly represented by a 128 dimensional byte vector



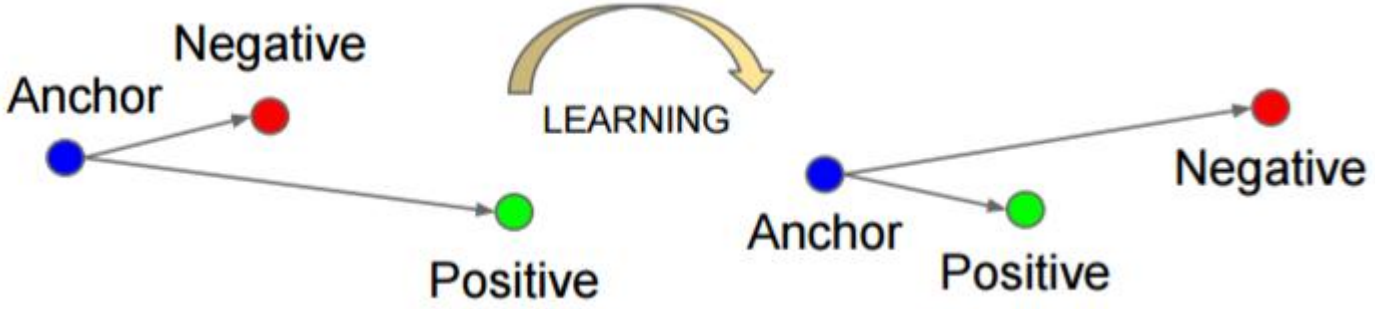


Distances of FaceNet embedding outputs
Threshold of 1.1 can classify each pair correctly
(Pose and illumination invariance)

FaceNet

- **Triplet Loss** (looking at 3 face images at a time)
  - embed an image x into a d-dimensional Euclidean space, called Embeddings *f(x)*.
  - To minimize Euclidean distance between an anchor and a positive (same) identity and maximize the distance between the anchor and a negative (different) identity.

$$\|x_i^a - x_i^p\|_2^2 + \alpha < \|x_i^a - x_i^n\|_2^2, \ \forall \left(x_i^a, x_i^p, x_i^n\right) \in \mathcal{T}$$



The loss to be minimized

$$\mathcal{L}_{\text{tri}}^m(x, x^+, x^-; f) = \max\left(0, \|f - f^+\|_2^2 - \|f - f^-\|_2^2 + \alpha\right)$$

FaceNet

- **Triplet Selection**
  - to ensure fast convergence
  - Hard positive
    - Find a x_i^p such that $\mathrm{argmax}_{x_i^p} \|f(x_i^a) - f(x_i^p)\|_2^2$
  - Hard negative
    - Find a x_i^n such that $\mathrm{argmin}_{x_i^n} \|f(x_i^a) - f(x_i^n)\|_2^2$
  - Hard to compute argmin and argmax in the whole training set, only compute them within a mini-batch.

FaceNet – Evaluation

- 100M-200M training face thumbnails with about 8M different identities.
- Result: **each face is compactly represented by a 128 dimensional byte vector**
  - During training a 128-D float vector is used, but it can be quantized to 128-bytes without loss of accuracy.
  - Smaller embeddings are possible at a minor loss of accuracy and could be employed on mobile devices
- **Face verification**: given a pair of two face images, a **L2 distance threshold** is used to determine whether they are same or different persons.
  - No model is trained specificly for verification task

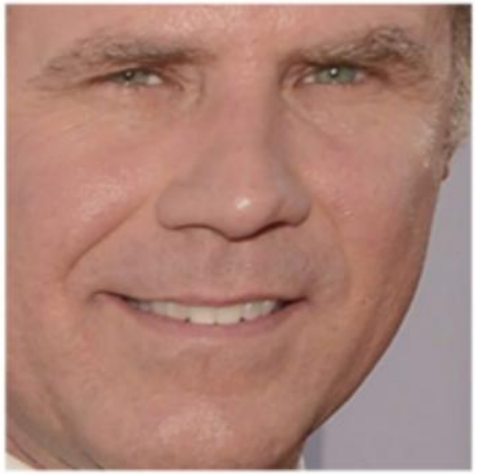
#dims	VAL
64	$86.8\% \pm 1.7$
128	$87.9\% \pm 1.9$
256	$87.7\% \pm 1.9$
512	$85.6\% \pm 2.0$

Effect of Embedding dimensionality

FaceNet – Embeddings example

- OpenFace provides a script that will generate embeddings of images

Input Image



128 Measurements Generated from Image

0.097496084868908	0.045223236083984	-0.1281466782093	0.032084941864014
0.12529824674129	0.060309179127216	0.17521631717682	0.020976085215807
0.030809439718723	-0.01981477253139	0.10801389068365	-0.00052163278451189
0.036050599068403	0.065554238855839	0.0731306001544	-0.1318951100111
-0.097486883401871	0.1226262897253	-0.029626874253154	-0.0059557510539889
-0.0066401711665094	0.036750309169292	-0.15958009660244	0.043374512344599
-0.14131525158882	0.14114324748516	-0.031351584941149	-0.053343612700701
-0.048540540039539	-0.061901587992907	-0.15042643249035	0.078198105096817
-0.12567175924778	-0.10568545013666	-0.12728653848171	-0.076289616525173
-0.061418771743774	-0.074287034571171	-0.065365232527256	0.12369467318058
0.046741496771574	0.0061761881224811	0.14746543765068	0.056418422609568
-0.12113650143147	-0.21055991947651	0.0041091227903962	0.089727647602558
0.061606746166945	0.11345765739679	0.021352224051952	-0.0085843298584223
0.061989940702915	0.19372203946114	-0.086726233363152	-0.022388197481632
0.10904195904732	0.084853030741215	0.09463594853878	0.020696049556136
-0.019414527341723	0.0064811296761036	0.21180312335491	-0.050584398210049
0.15245945751667	-0.16582328081131	-0.035577941685915	-0.072376452386379
-0.12216668576002	-0.0072777755558491	-0.036901291459799	-0.034365277737379
0.083934605121613	-0.059730969369411	-0.070026844739914	-0.045013956725597
0.087945111095905	0.11478432267904	-0.089621491730213	-0.013955107890069
-0.021407851949334	0.14841195940971	0.078333757817745	-0.17898085713387
-0.018298890441656	0.049525424838066	0.13227833807468	-0.072600327432156
-0.011014151386917	-0.051016297191381	-0.14132921397686	0.0050511928275228
0.0093679334968328	-0.062812767922878	-0.13407498598099	-0.014829395338893
0.058139257133007	0.0048638740554452	-0.039491076022387	-0.043765489012003
-0.024210374802351	-0.11443792283535	0.071997955441475	-0.012062266469002
-0.057223934680223	0.014683869667351	0.05228154733777	0.012774495407939
0.023535015061498	-0.081752359867096	-0.031709920614958	0.069833360612392
-0.0098039731383324	0.037022035568953	0.11009479314089	0.11638788878918
0.020220354199409	0.12788131833076	0.18632389605045	-0.015336792916059
0.0040337680839002	-0.094398014247417	-0.11768248677254	0.10281457751989
0.051597066223621	-0.10034311562777	-0.040977258235216	-0.082041338086128

FaceNet: Performance

- **LFW** (Labeled Faces in the Wild)
  - 9 training splits are used to select L2-distance threshold, test is then performed on the 10th test split. (The selected threshold is 1.242)
  - Model1: Fixed center crop of the LFW thumbnail.
    - Accuracy: **98.87**%
  - Model2: Use a proprietary face detector
    - Accuracy: **99.63**%
- **YTF**(Youtube Faces DB)
  - Accuracy: **95.12%** (DeepFace: 91.4%)

DeepFace on LFW

DeepFace-single	**0.9592**	unsupervised
DeepFace-single	**0.9700**	restricted
DeepFace-ensemble	**0.9715**	restricted
DeepFace-ensemble	**0.9735**	unrestricted

FaceNet: Contribution

- Use Triplet instead of Softmax
  - Triplet loss only needs low-D feature vectors (128 bytes), Softmax often needs large-D feature vectors (4k in many models)
- No need face alignment
  - DeepFace needs alignment
- Can do verification directly using L2 distance on 128-D vectors
  - DeepFace uses another classification model (SVM) on feature vectors
- Perform the best in the public dataset
  - **LFW, YTF**

FaceNet: Result – LFW errors

- False accept cases

FaceNet: Result – LFW errors

- False reject

Vector Distance

1. Euclidean distance (L2)

$$d = \sqrt{(V_1 - V_2)^2}$$

2. Dot-product similarity

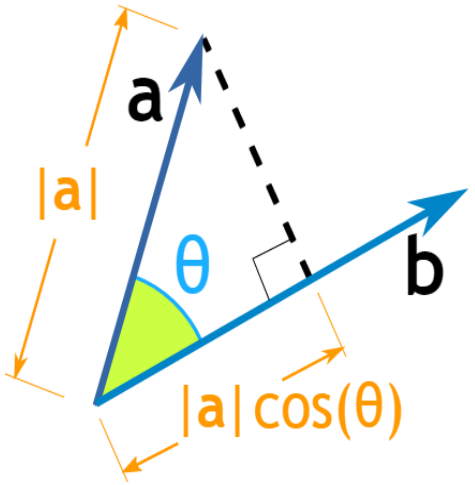
$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$

- Measure the **similarity**
- The term cos(θ) captures how align two vectors are

Relationship	Angle θ	$\cos(\theta)$	Dot product
Identical direction	0°	1	Large positive
Perpendicular	90°	0	0
Opposite direction	180°	-1	Large negative

- The result is a unbounded scalar .

Vector Distance



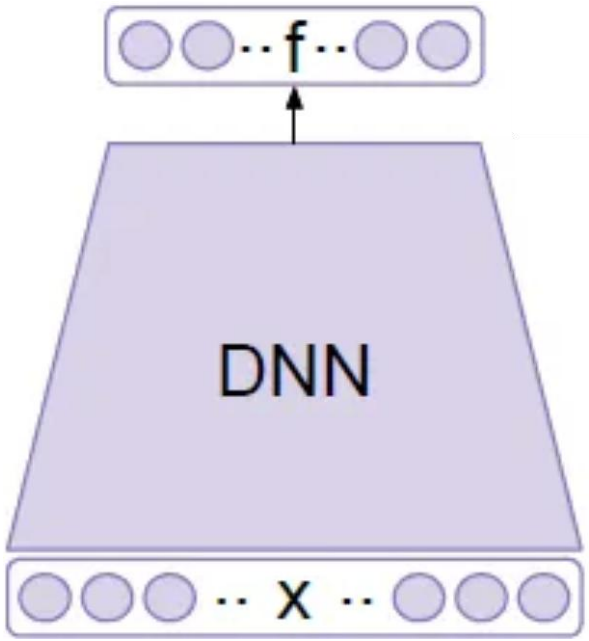
3. Cosine similarity $\cos(\theta)$

- Inner product of two vectors a, b can be interpreted as projecting a to b, and then taking product of projected a length with b length
- Cosine similarity is a *normalized* inner product (dot product)

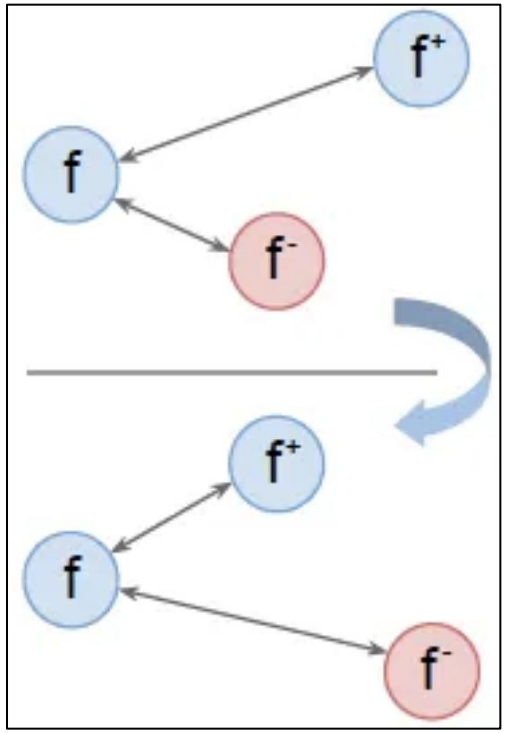
$$\vec{a} \bullet \vec{b} = |a||b|\cos(\theta) \quad \Rightarrow \quad cos(\theta) = \frac{\vec{a} \bullet \vec{b}}{|a||b|}$$

- If we L2-normalize both vectors, dot product becomes cosine similarity
  - Only care about the **angle** of two vectors, independent of their magnitudes.
- Cosine similarity $\cos(\theta)$ is a bounded scalar in between [-1, 1]

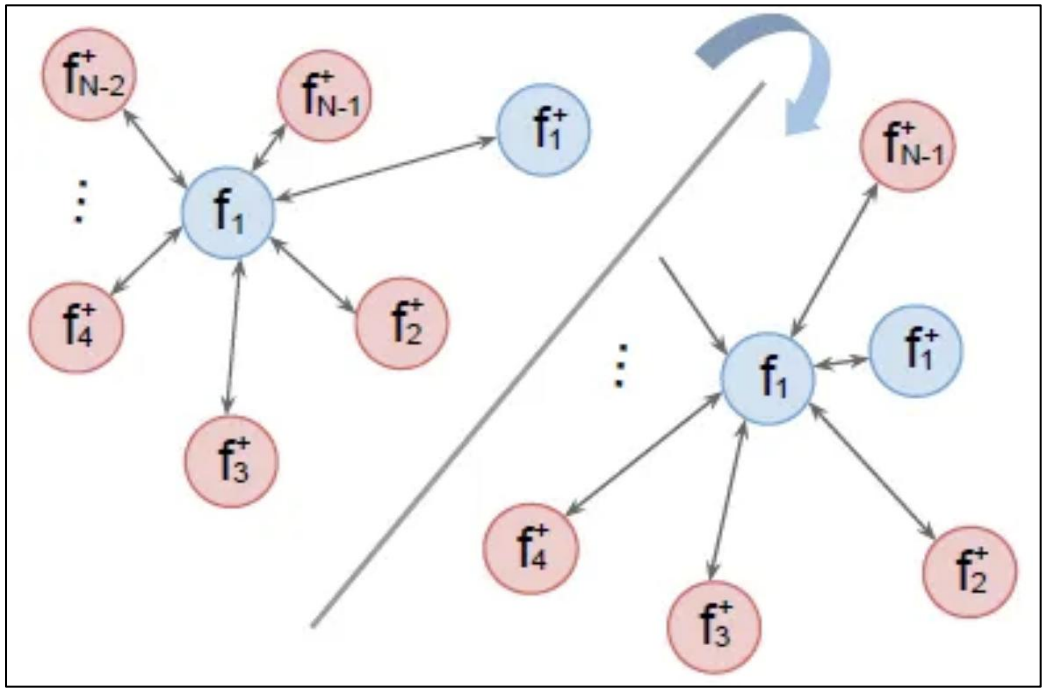
Tuplet Loss (Improved Deep Metric Learning with Multi-class N-pair Loss Objective)



Triplet Loss



Tuplet Loss



Tuplet Loss

- **(N+1)-tuplet loss for multiple negative samples**
  - pushes N-1 negative examples all at once
  - For anchor x, $x+$ is a positive example, and $\{x_1, \ldots, x_{N-1}\}$ are negative.

We hope it << 0

$$\mathcal{L}(\{x, x^+, \{x_i\}_{i=1}^{N-1}\}; f) = \log\left(1 + \sum_{i=1}^{N-1} \exp(f^\top f_i - f^\top f^+)\right)$$

To minimize

To maximize

  - The ideal loss is to compare each anchor against *all* negative samples in dataset. (N+1)-tuplet loss approximates it by using N sampled negatives.

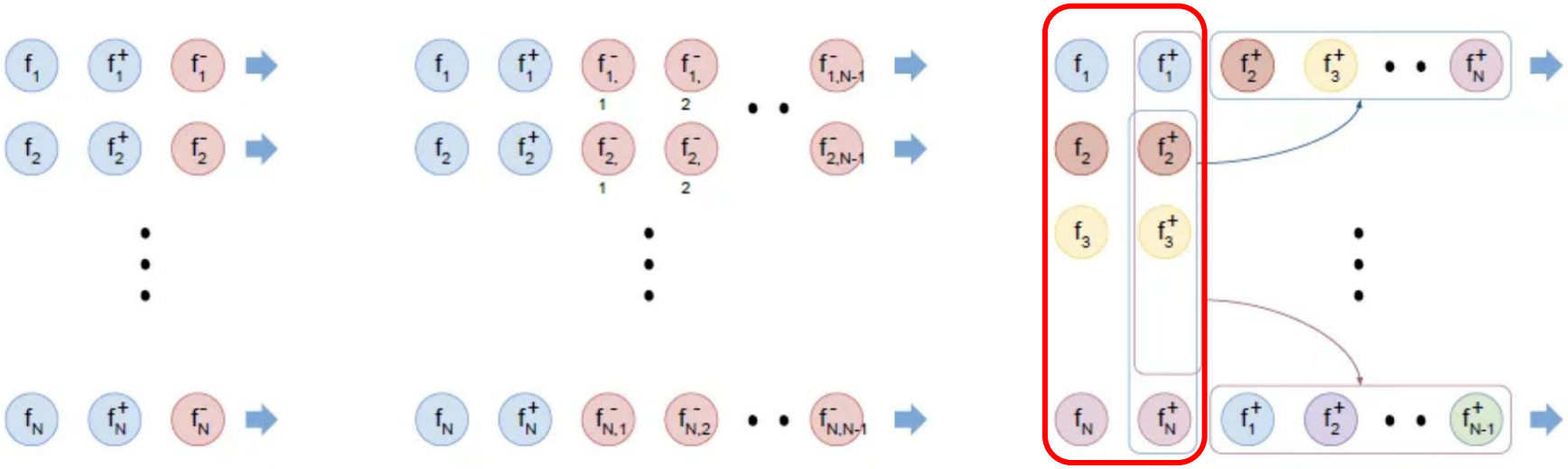
Tuplet Loss

- **N-pair Batch Construction**
  - A mini-batch contains N pairs
    - each pair : 1 anchor + 1 positive
    - each anchor has N-1 negative samples

$$L_i = \log\left(1 + \sum_{j \neq i} \exp(x_i^\top x_j - x_i^\top x_i^+)\right)$$

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i$$

N-pairs (2N samples in a mini-batch)



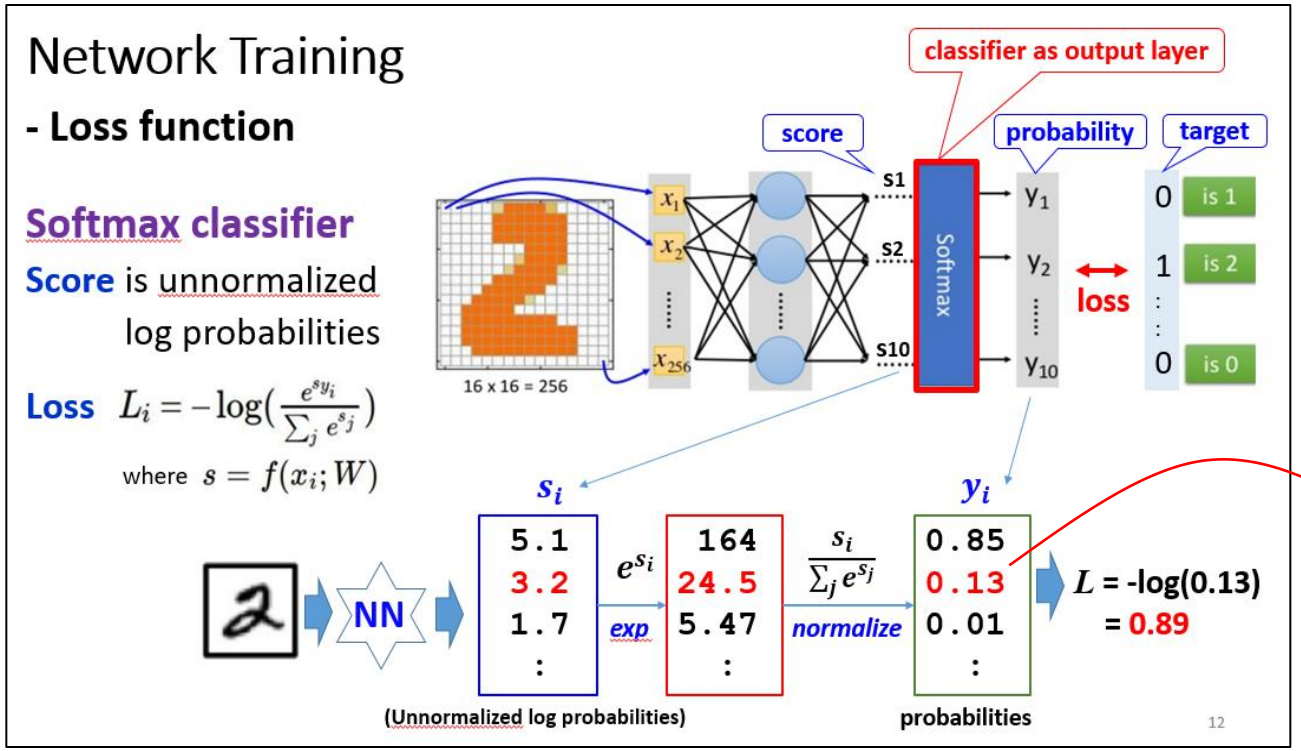
(a) Triplet loss (b) $(N+1)$-tuplet loss (c) N-pair-mc loss

Tuplet Loss

To maximize positive pair similarity
and minimize negative pair similarity

- **N-pair multi-class loss**
  - derived from **Softmax loss**

$$L_i = \log\left(1 + \sum_{j \neq i} \exp(x_i^\top x_j - x_i^\top x_i^+)\right)$$



$$L_i = -\log(p_i)$$

To maximize positive pair similarity

$$p_i = \frac{\exp(x_i^\top x_i^+)}{\exp(x_i^\top x_i^+) + \sum_{j \neq i} \exp(x_i^\top x_j)}$$

positive pair negative pairs

From Softmax loss to N-pair-mc loss

$$L_i = -\log(p_i)$$

Softmax

$$L_i = \boxed{-\log \frac{\exp(x_i^\top x_i^+)}{\exp(x_i^\top x_i^+) + \sum_{j \neq i} \exp(x_i^\top x_j)}}$$

$$\log \frac{X}{Y} = \log X - \log Y$$

$$\log e^a = a$$

$$L_i = \log \left(\exp(x_i^\top x_i^+) + \sum_{j \neq i} \exp(x_i^\top x_j) \right) - x_i^\top x_i^+$$

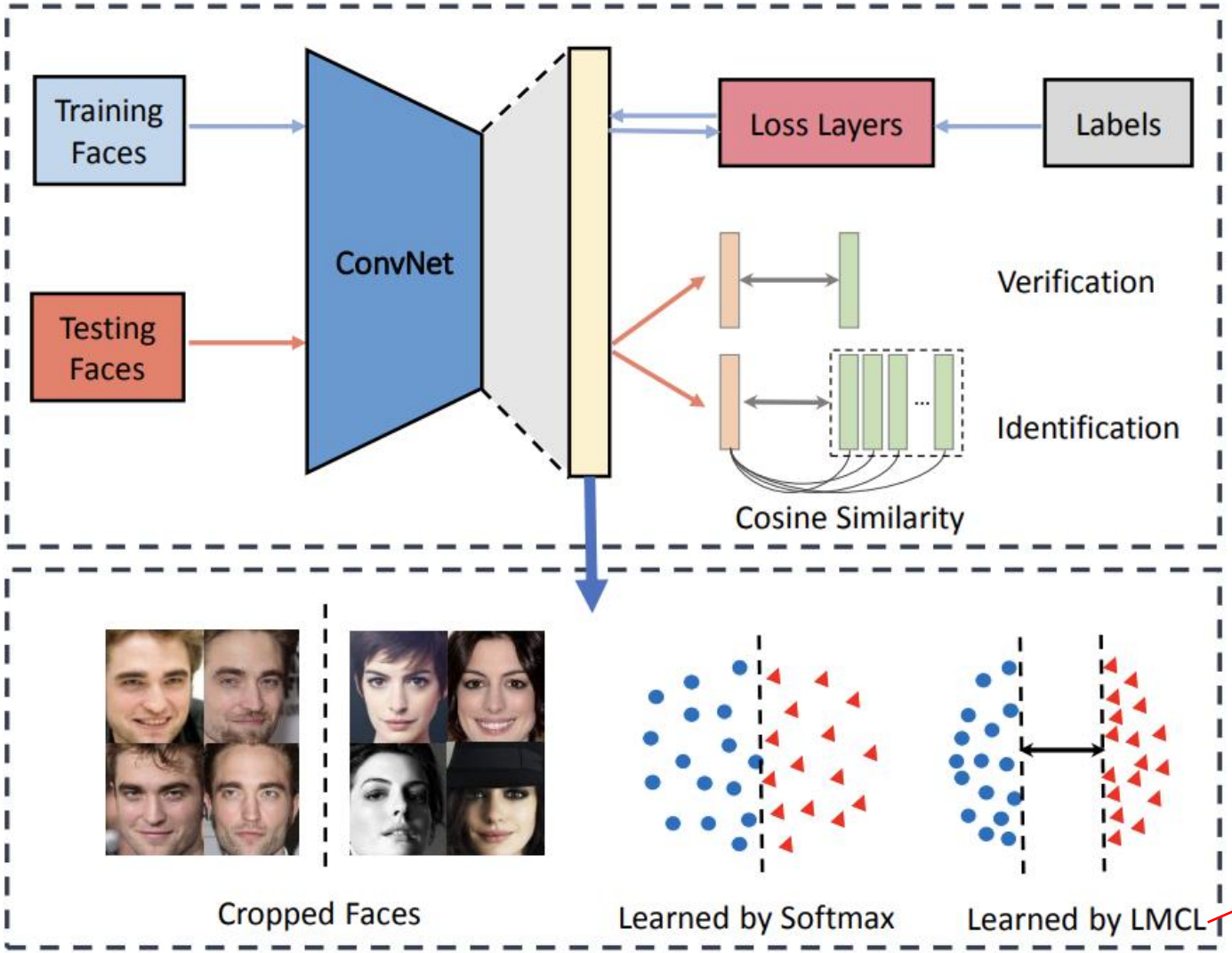
$$L_i = \log \left[\exp(x_i^\top x_i^+) \left(1 + \sum_{j \neq i} \exp(x_i^\top x_j - x_i^\top x_i^+) \right) \right] - x_i^\top x_i^+$$

$$\log(ab) = \log a + \log b$$

$$L_i = \cancel{x_i^\top x_i^+} + \boxed{\log \left(1 + \sum_{j \neq i} \exp(x_i^\top x_j - x_i^\top x_i^+) \right)} - \cancel{x_i^\top x_i^+}$$

N-pair multi-class loss

CosFace: Large Margin Cosine Loss for Deep Face Recognition



Large Margin Cosine Loss

CosFace

- **Softmax loss** $$L = -\log \frac{e^{W_{y_i}^T x_i + b_{y_i}}}{\sum_{j=1}^{N} e^{W_j^T x_i + b_j}}$$

$$W_j^T x = \|W_j\| \|x\| \cos\theta_j$$

N : number of classes

- Fix bias $b_j = 0$ and normalize x and w using L2 norm, rescale $\|x\|$ to s

$$L = -\log \frac{e^{s\cos\theta_{y_i}}}{e^{s\cos\theta_{y_i}} + \sum_{j=1, j\neq y_i}^{N} e^{s\cos\theta_j}}$$

GT class, the larger the better

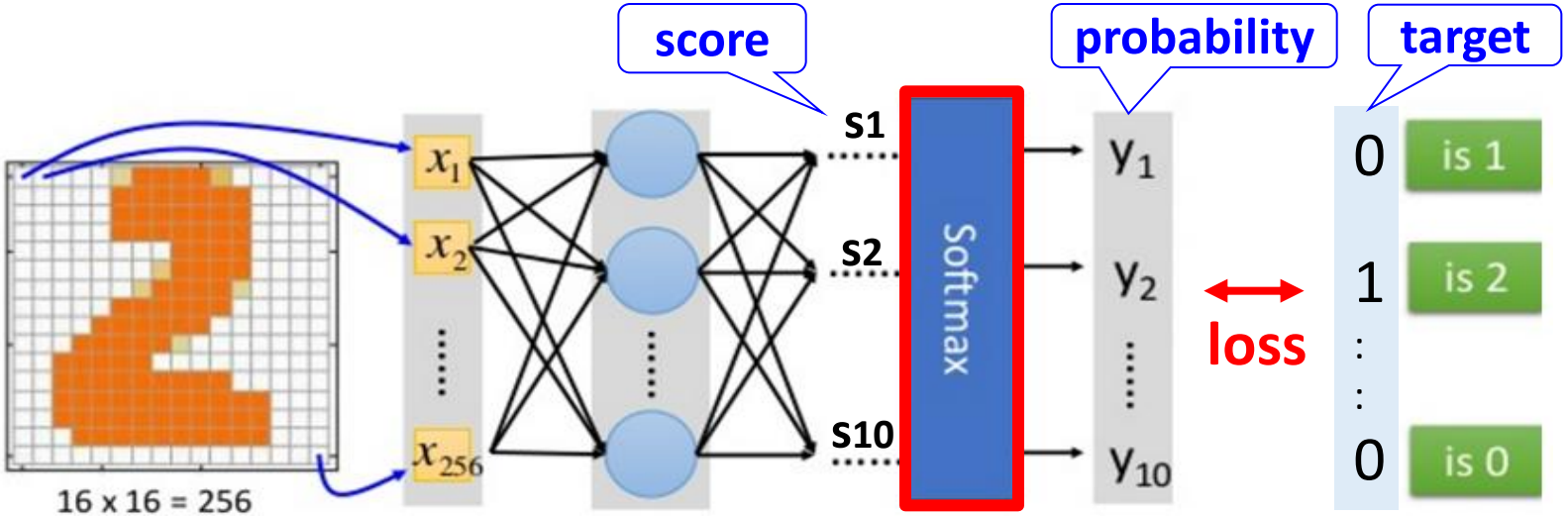
GT class

- **Large Margin Cosine Loss (LMCL)**
  - Add penalty to GT-class similarity

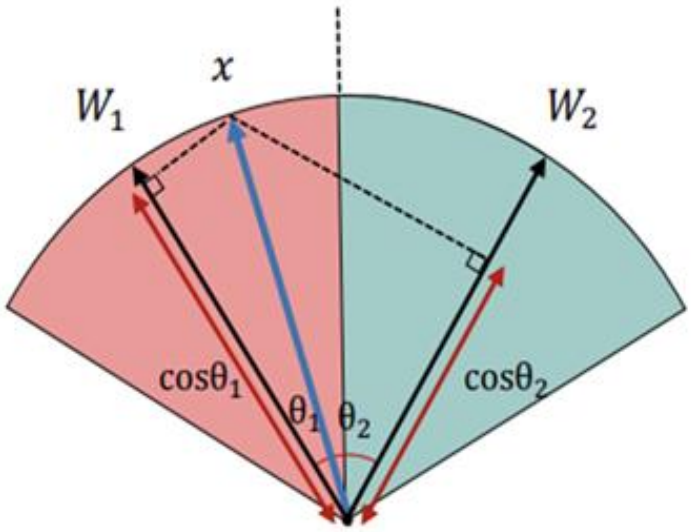
$$L = -\log \frac{e^{s(\cos\theta_{y_i} - m)}}{e^{s(\cos\theta_{y_i} - m)} + \sum_{j=1, j\neq y_i}^{N} e^{s\cos\theta_j}}$$

model needs to enlarge the cos similarity with GT class to compensate the penalty

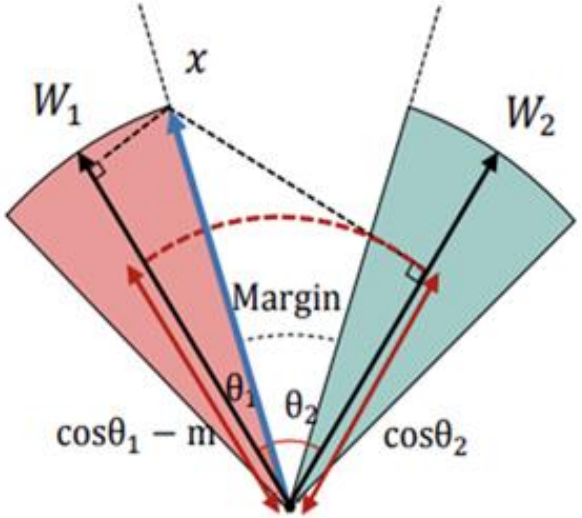
CosFace



$$\vec{x} \cdot \overrightarrow{w_1} = s_1 \quad (= cos\theta_{x,w_1})$$
$$\vec{x} \cdot \overrightarrow{w_2} = s_2$$
$$\vdots$$



NSL



LMCL

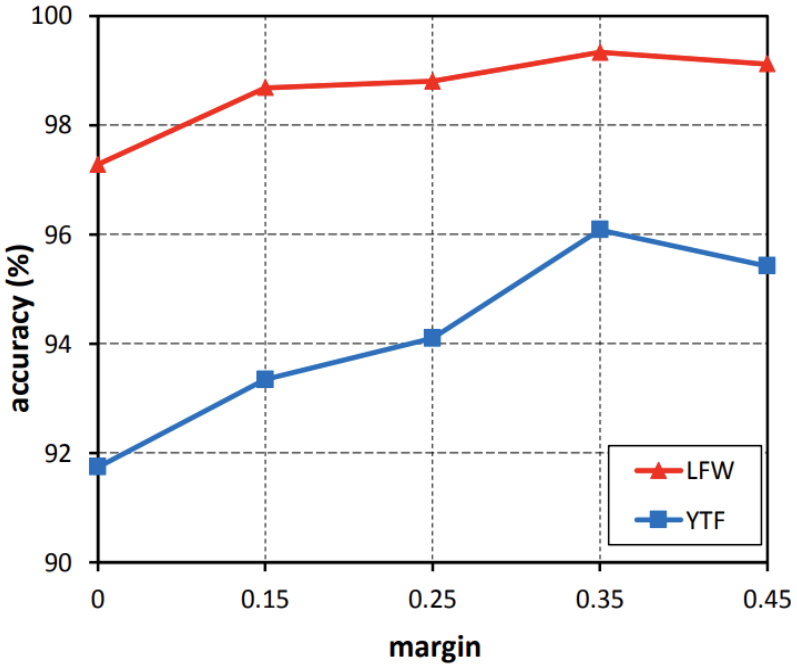
- We can regard $\overrightarrow{w_i}$ as the center vector of $class_i$
- Model is trained to learn the center vector of each class

CosFace

Method	Training Data	#Models	LFW	YTF
Deep Face[35]	4M	3	97.35	91.4
FaceNet[29]	200M	1	99.63	95.1
DeepFR [27]	2.6M	1	98.95	97.3
DeepID2+[33]	300K	25	99.47	93.2
Center Face[42]	0.7M	1	99.28	94.9
Baidu[21]	1.3M	1	99.13	-
SphereFace[23]	0.49M	1	99.42	95.0
CosFace	5M	1	**99.73**	**97.6**

Triplet loss

identity loss
with margin



Effect of Margin (m)

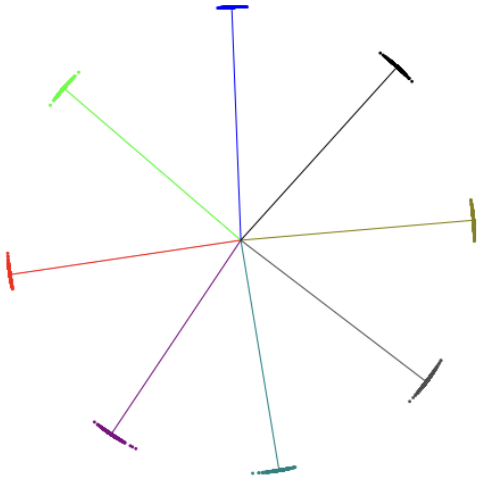
ArcFace: Additive Angular Margin Loss for Deep Face Recognition

- **Angular Margin**

Add margin to angle, instead of cosθ

$$-\log \frac{e^{s\cos(\theta_{y_i}+m)}}{e^{s\cos(\theta_{y_i}+m)}+\sum_{j=1,j\neq y_i}^{N} e^{s\cos\theta_j}}.$$

(a) Norm-Softmax



(b) ArcFace

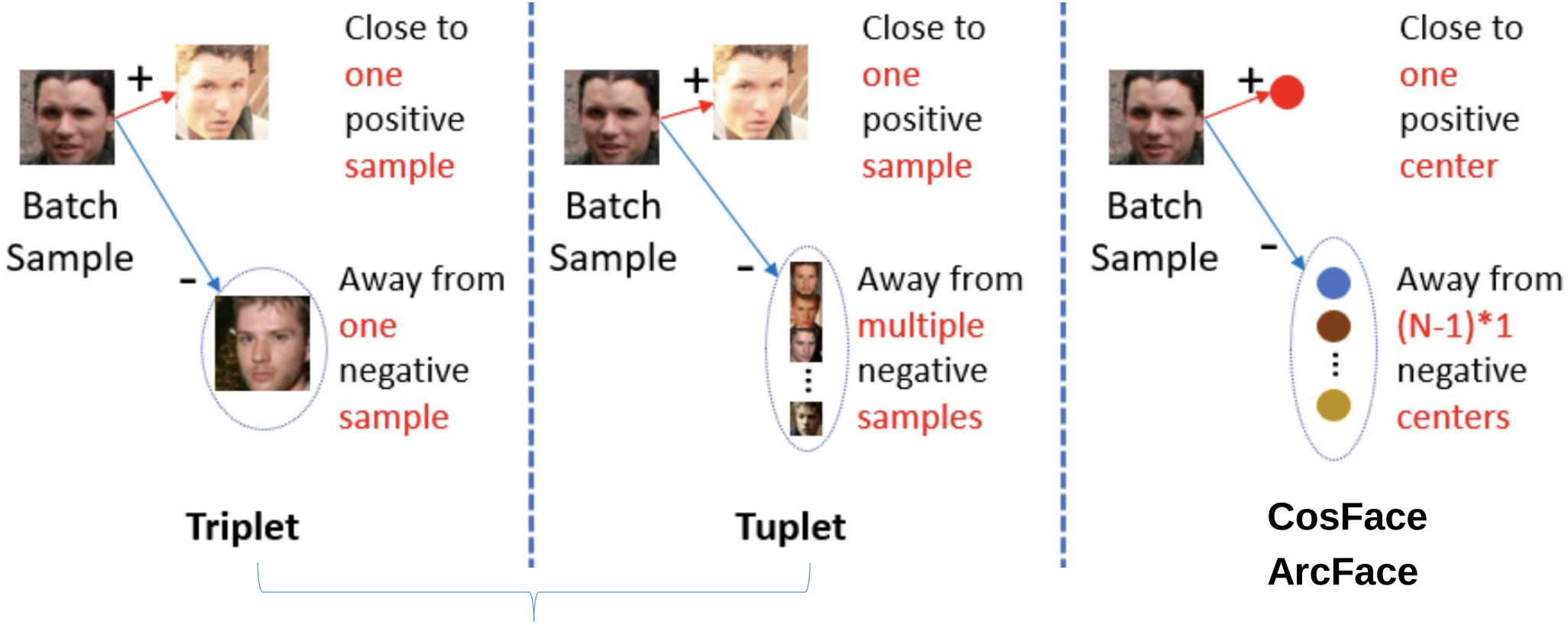
Better interpretation, better performance

Method	#Image	LFW	YTF
ArcFace	5.1M	99.83	98.02
ArcFace	11.96M	99.83	98.01

Review

Deepface: identity loss

DeepID: identity loss + contrastive loss



Local comparison

Global comparison